

Lớp: IT003.R17

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: (MSSV:)

Nội dung báo cáo:

I. Kết quả thử nghiệm

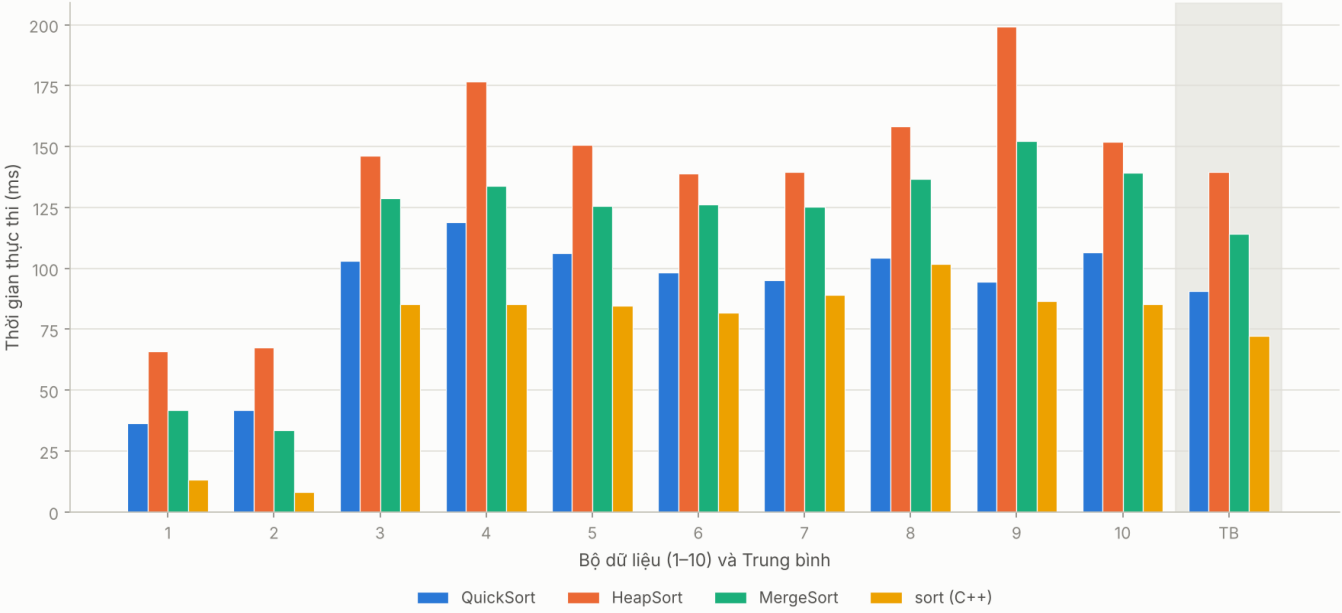
1. Bảng thời gian thực hiện

Dữ liệu	Thời gian thực hiện (ms)			
	QuickSort	HeapSort	MergeSort	sort (C++)
1	36.44	66.06	41.70	13.35
2	41.75	67.59	33.48	8.20
3	103.12	146.34	128.74	85.22
4	118.97	176.78	133.87	85.23
5	106.15	150.56	125.67	84.71
6	98.43	138.97	126.22	81.91
7	95.16	139.41	125.28	89.23
8	104.41	158.27	136.70	101.92
9	94.51	199.09	152.10	86.62
10	106.50	151.81	139.13	85.36
Trung bình	90.54	139.49	114.29	72.17

Môi trường thử nghiệm: biên dịch g++ -O2 -std=c++17; mỗi dãy ~1.000.000 số thực (kiểu double); dữ liệu sinh ngẫu nhiên với seed cố định = 2026 để đảm bảo khả năng tái lập.

2. Biểu đồ (cột) thời gian thực hiện

So sánh thời gian thực thi của QuickSort, HeapSort, MergeSort và std::sort
(mỗi dãy ~1.000.000 số thực)



II. Kết luận

Từ bảng số liệu và biểu đồ thực nghiệm trên 10 bộ dữ liệu (mỗi bộ ~1.000.000 số thực), có thể rút ra một số nhận xét sau:

- `std::sort` (C++) cho thời gian thực thi trung bình thấp nhất (~72 ms) trong cả 10 bộ dữ liệu, kể cả với dữ liệu đã có thứ tự (tăng/giảm dần) lẫn dữ liệu ngẫu nhiên. Điều này phù hợp với lý thuyết vì `std::sort` trong thư viện chuẩn C++ cài đặt thuật toán Introsort — kết hợp QuickSort, HeapSort và Insertion Sort, cùng nhiều tối ưu hóa ở mức mã máy (cache locality, loop unrolling, tránh đệ quy sâu...) mà một cài đặt thủ công khó đạt được.
- QuickSort (tự cài đặt, chọn pivot ngẫu nhiên) có thời gian thực thi trung bình đứng thứ 2 (~90 ms), nhanh hơn rõ rệt trên dữ liệu đã có thứ tự (bộ 1, 2: ~36–42 ms) so với dữ liệu ngẫu nhiên (~95–119 ms). Nhờ chọn pivot ngẫu nhiên, thuật toán tránh được trường hợp xấu nhất $O(n^2)$ thường gặp khi dùng pivot cố định trên dữ liệu đã sắp xếp.
- MergeSort có thời gian ổn định hơn giữa các bộ dữ liệu (dao động ít), do độ phức tạp luôn là $O(n \log n)$ bất kể thứ tự đầu vào, nhưng tốn thêm bộ nhớ phụ $O(n)$ cho việc trộn (merge), nên nhìn chung chậm hơn QuickSort trên dữ liệu ngẫu nhiên.
- HeapSort có thời gian thực thi trung bình cao nhất (~139 ms) dù độ phức tạp lý thuyết cũng là $O(n \log n)$. Nguyên nhân chủ yếu là do heap truy cập bộ nhớ không liên tục (non-sequential memory access khi sift-down giữa các mức của cây nhị phân), gây nhiều cache-miss hơn so với MergeSort và QuickSort vốn duyệt mảng tuần tự hơn.
- Với 2 bộ dữ liệu đã có thứ tự sẵn (bộ 1 tăng dần, bộ 2 giảm dần), tất cả 4 thuật toán đều chạy nhanh hơn đáng kể so với 8 bộ dữ liệu ngẫu nhiên còn lại, đặc biệt là `std::sort` và MergeSort giảm hơn 4–6 lần thời gian thực thi.

Tóm lại, kết quả thực nghiệm phù hợp với độ phức tạp lý thuyết $O(n \log n)$ của cả 4 thuật toán, đồng thời cho thấy rõ ảnh hưởng của yếu tố cài đặt thực tế (cache locality, hằng số ẩn trong O -lớn) đến hiệu năng thực thi, chứ không chỉ phụ thuộc vào độ phức tạp tiệm cận.

III. Thông tin chi tiết – link GitHub

Toàn bộ mã nguồn, dữ liệu thử nghiệm và báo cáo được lưu trữ công khai (public) tại:

[GITHUB URL](#)

Repository bao gồm:

1. Báo cáo (file PDF báo cáo kết quả thử nghiệm này)
2. Mã nguồn (chương trình sinh dữ liệu, cài đặt 4 thuật toán sắp xếp, chương trình đo thời gian thực thi, script vẽ biểu đồ)
3. Dữ liệu thử nghiệm (10 bộ dữ liệu nhị phân đã sinh, và file kết quả CSV)